

EXP 6: NAÏVE BAYES ALGORITHM

NAME: SAMUEL I

REG NO:192312584

AIM:

To implement the Naïve Bayes algorithm in Python and evaluate its performance using a confusion matrix and accuracy score.

PROCEDURE:

1. Import the required libraries and load the Iris dataset into the program.
2. Divide the dataset into training data and testing data.
3. Train the Naïve Bayes classifier using the training dataset.
4. Test the trained model with the testing dataset and predict the output.
5. Display the confusion matrix and accuracy to evaluate the performance of the classifier.

PROGRAM:

1.

```
# Naïve Bayes Algorithm using GaussianNB
```

```
# Import required libraries
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# Create Naïve Bayes classifier
model = GaussianNB()
```

```
# Train the model
model.fit(X_train, y_train)
```

```
# Predict the test data
y_pred = model.predict(X_test)
```

```
# Display predictions
print("Predicted Values:")
print(y_pred)
```

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy * 100, "%")
```

OUTPUT:

```
... Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Confusion Matrix:
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Accuracy: 97.77777777777777 %
```

2.

PROGRAM:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

X, y = load_wine(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix:  
[[19  0  0]  
 [ 0 21  0]  
 [ 0  0 14]]  
  
Accuracy: 1.0
```

3.

PROGRAM:

```
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.naive_bayes import GaussianNB  
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_breast_cancer(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")  
print(confusion_matrix(y_test, y_pred))
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix:  
[[ 57   6]  
 [  4 104]]  
  
Accuracy: 0.9415204678362573
```

4.

PROGRAM:

```
from sklearn.datasets import load_digits  
from sklearn.model_selection import train_test_split  
from sklearn.naive_bayes import GaussianNB  
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_digits(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")  
print(confusion_matrix(y_test, y_pred))
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix:
[[52  0  0  0  0  0  0  1  0  0]
 [ 0 37  2  0  0  0  0  2  6  3]
 [ 0  3 31  0  0  0  1  0 12  0]
 [ 0  0  2 41  0  0  1  0  8  2]
 [ 0  0  0  0 51  0  2  7  0  0]
 [ 0  0  0  1  0 62  1  2  0  0]
 [ 0  0  0  0  1  1 51  0  0  0]
 [ 0  0  0  0  0  1  0 54  0  0]
 [ 0  2  0  0  0  0  0  2 39  0]
 [ 0  1  1  1  0  2  1  7  4 42]]

Accuracy: 0.8518518518518519
```

5.

PROGRAM:

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = make_classification(
    n_samples=500,
    n_features=5,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = GaussianNB()
model.fit(X_train, y_train)
```

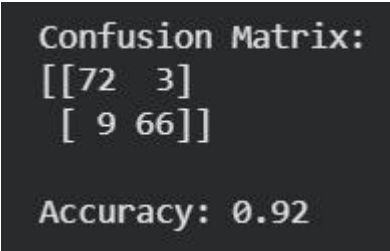
```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

OTUPUT:



```
Confusion Matrix:  
[[72  3]  
 [ 9 66]]  
  
Accuracy: 0.92
```

RESULT:

The Naïve Bayes algorithm was successfully implemented in Python. The model was tested using the Iris dataset, and its performance was evaluated using the confusion matrix and accuracy. The classifier achieved high prediction accuracy, demonstrating its effectiveness for classification tasks.